HEMOVAULT
CENTRALIZED BLOOD BANK MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted by

PRIYAN R 310624148072

in partial fulfillment of the requirements for the award of the Degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING (ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

2311CSE503R - FULL STACK DEVELOPMENT



EASWARI ENGINEERING COLLEGE

An Autonomous Institution affiliated to Anna University

Chennai

OCTOBER 2026

BONAFIDE CERTIFICATE

EASWARI ENGINEERING COLLEGE

Certified that this mini project report titled "HemoVault Centralized Blood Bank Management System" is the bonafide work of Priyan R (Registration Number 310624148072), Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning), who carried out the project work under my supervision.

HEAD OF THE DEPARTMENT

Dr. Y. Justin Dhas

Department of CSE(AIML)

Easwari Engineering College

Signature: _____

SUPERVISOR

Ms Divyanjali

Project Guide

Department of CSE(AIML)

Signature: _____

Place: Chennai

Date: _____

CERTIFICATE OF VIVA VOCE EXAMINATION

EASWARI ENGINEERING COLLEGE

This is to certify that the Viva Voce Examination of the project work entitled "HemoVault Centralized Blood Bank Management System", submitted by Mr. Priyan R (Registration Number 310624148072), in partial fulfillment of the requirements for the award of the degree of Bachelor of Engineering, was conducted on _____ in the Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning), Easwari Engineering College, Ramapuram, Chennai.

INTERNAL EXAMINER

Name: _____

Designation: _____

Address: _____

Signature: _____

EXTERNAL EXAMINER

Name: _____

Designation: _____

Address: _____

Signature: _____

ABSTRACT

Healthcare institutions increasingly manage critical transfusion resources through web applications, but a browser-based blood bank platform is dependable only when the system controls who may requisition blood products, how long inventory units remain viable in cold storage, how donor pathology tests are evaluated, and how emergency cross-matching requests are fulfilled. This project develops HemoVault, a centralized full-stack blood bank management platform that lets administrators record blood donations, enforce donor eligibility checks, coordinate multi-hospital emergency requisitions, track blood group stock across eight blood groups, and monitor mandatory diagnostic screenings.

The system uses Node.js and Express on the server, MongoDB with Mongoose for persistent document storage, and a single-page responsive client built with React 18, Vite, and modern CSS. Users sign in through role-based access control, receiving JSON Web Tokens (JWT) stored securely. Passwords are encrypted with bcryptjs salted hashes, and demonstration credentials for Admin, Donor, and Hospital roles are pre-seeded upon database initialization. Every API route is guarded by authentication middleware, and administrative actions are protected by strict role authorization checks.

A blood donation record represents a physical whole blood or component unit whose expiration date is automatically calculated on the server based on blood component preservation rules (42 days for packed red blood cells, 5 days for platelets, and 365 days for fresh frozen plasma). Before a donation is finalized, donors complete a digital health questionnaire and vitals check. During laboratory processing, pathologists record five statutory screening markers: HIV, Hepatitis B, Hepatitis C, Syphilis, and Malaria. Units failing any test are automatically quarantined and marked for safe biohazard disposal.

Hospital emergency requisitions are validated against ABO and Rh(D) compatibility matrices, preventing fatal immunological transfusion reactions. When stock is allocated, unit quantities decrement atomically to avoid double-allocation anomalies. The administrative portal provides live inventory matrices, low-stock threshold alerts, expiration tracking, donor retention metrics, and audit logs. The repository includes automated API route tests and a Vercel serverless deployment entry point. The report documents requirements, architecture, data schemas, API contracts, testing, and operational procedures.

Keywords: blood bank management, transfusion safety, MERN stack, donor eligibility, component preservation, infectious disease screening, hospital emergency requisition, role-based access control

ACKNOWLEDGEMENT

I express my sincere gratitude to the Principal and Management of Easwari Engineering College for giving me the opportunity and the facilities to carry out this project work.

I convey my heartfelt thanks to Dr. Y. Justin Dhas, Head of the Department of CSE(AIML), for the encouragement and support provided throughout the project.

The guidance, suggestions and motivation of my project guide, Ms Divyanjali, Department of CSE(AIML), throughout the development of the HemoVault Centralized Blood Bank Management System were of great help.

I extend my gratitude to the faculty members of the Department of CSE(AIML) for their academic support. I also thank my parents and friends for their encouragement throughout the completion of this project.

Priyan R

310624148072

TABLE OF CONTENTS

TITLE	PAGE
ABSTRACT	**4**
ACKNOWLEDGEMENT	**5**
1 INTRODUCTION	**7**
1.1 Overview	7
1.2 Problem Definition	7
1.3 Objective	8
1.4 Scope of the Project	8
1.5 Motivation	8
1.6 Stakeholders and Operational Responsibilities	9
1.7 Blood Safety and Inventory Concepts	10
2 RELATED WORK	**11**
2.1 Literature Review	11
2.2 Existing System	12
2.3 Proposed System	12
2.4 Comparison of Blood Banking Approaches	13
2.5 Design Lessons and System Positioning	14
3 SYSTEM ANALYSIS	**15**
3.1 System Requirements	15
3.2 Software and Hardware Requirements	16
3.3 Functional Use Cases and Acceptance Conditions	17
3.4 Access Control and Hospital Verification	18
3.5 Data Quality and Transfusion Safety Rules	19
4 SYSTEM DESIGN	**20**
4.1 System Architecture	20
4.2 Module Description	21
4.3 Data Model and Entity Isolation	21
4.4 Database Diagram	22
4.5 Sequence Diagram	23
4.6 Data Flow and Request Processing	24
4.7 Table Design and Record Relationships	25
4.8 REST Interface and Response Contracts	26
4.9 Interface State and Feedback Design	27
5 IMPLEMENTATION	**28**
5.1 Implementation Overview	28
5.2 Key Functions and Code Explanation	29
5.3 End to End Donor Registration & Donation Workflow	32
5.4 Hospital Emergency Requisition Workflow	33
5.5 Worked Example of Blood Matching and Allocation	34
5.6 Cold Storage & Expiration Lifecycle	35
5.7 Deployment Configuration and Persistence	36
5.8 Security Hardening Measures	37
5.9 Test Coverage and Regression Traceability	38
5.10 Operating Guide and Troubleshooting	39
6 CONCLUSION AND FUTURE WORK	**40**
6.1 Conclusion	40
6.2 Future Enhancements	40
6.3 Evaluation Limits and Enhancement Priorities	41
APPENDIX SOURCE CODE	**42**

CHAPTER 1
INTRODUCTION

1.1 Overview

Centralized blood banking replaces fragile manual paper ledgers, fragmented hospital logs, and uncoordinated telephone communications with a unified digital repository, real-time inventory tracking, and an automated allocation workflow. That transformation makes the blood donation and requisition process fast, transparent, and auditable, but it also transfers critical clinical responsibilities to software systems. The platform must verify who may requisition blood products, maintain strict temperature and shelf-life tracking so that compromised units are never transfused, validate immunological compatibility without trusting client-side inputs, and record every handling event so hospital blood bank officers can review cold-chain compliance.

HemoVault is a full stack centralized blood bank management system engineered for this clinical purpose. Administrators sign in to inspect real-time blood stock across all eight ABO and Rh(D) blood groups, coordinate emergency hospital requisitions, record laboratory infectious disease screenings, and view inventory turnover metrics. Donors sign in to verify clinical eligibility, view regional donation drives, book phlebotomy appointments, and track their donation history. Partner hospitals submit urgent unit requests with specified urgency tiers and track delivery dispatch. The application is developed with Node.js and Express on the backend, MongoDB for schema-flexible document storage, and React 18 with Vite for a fast, component-driven user interface.

1.2 Problem Definition

Traditional blood banks in semi-urban and regional clusters rely heavily on manual registers and disparate spreadsheets, creating severe systemic vulnerabilities. Hand-written donor cards are prone to transcription mistakes, physical registers prevent multi-center stock visibility during life-threatening trauma incidents, and expiry dates are tracked through manual visual inspection, leading to tragic discarding of precious blood units. Furthermore, when hospitals face acute surgical shortages, staff spend precious hours telephoning nearby blood banks without knowing current compatibility or stock levels.

The problem addressed by this project is to provide a reliable, auditable, and centralized blood banking workflow in which donor identity, physiological eligibility, laboratory pathogen clearance, and emergency allocation are strictly enforced by the server. Real-time inventory balances must be updated atomically to prevent double-allocation anomalies, while audit logs maintain full chain-of-custody tracking. The system is designed to run seamlessly in modern cloud serverless environments while remaining lightweight enough for standalone local deployment.

1.3 Objective

The primary objective is to develop a reliable, centralized, and secure digital blood bank management platform for regional health networks. Authorized administrators must be able to manage inventory across all eight blood groups and review infectious disease laboratory tests. Donors must be able to register, verify eligibility criteria, and schedule donations. Partner healthcare facilities must be able to submit emergency blood requisitions and receive real-time compatibility allocations.

Objective	Implemented approach
Authenticate every user	JWT cryptographic tokens, bcryptjs password hashing, role authorization middleware
Separate responsibilities	Distinct Admin, Donor, and Hospital roles enforced by route-level access control
Track inventory in real time	Mongoose document models tracking units by blood group, component, and storage location
Enforce clinical eligibility	Server-side validation of age, weight, pulse, blood pressure, and inter-donation intervals
Ensure biological safety	Mandatory laboratory screening recording HIV, HBV, HCV, Syphilis, and Malaria
Automate emergency allocation	ABO/Rh compatibility engine matching donor units with patient requisitions

1.4 Scope of the Project

The project scope encompasses user authentication across three roles (Admin, Donor, Hospital), automated inventory classification for whole blood, packed red cells, platelets, and plasma, donor health questionnaires with dynamic eligibility scoring, a laboratory diagnostic screening portal, emergency requisition management with urgency prioritization, audit logging of all unit transfers, and responsive user interfaces. The system can be deployed on local workstations or cloud platforms such as Vercel. Physical barcode scanning hardware, automated centrifuge control, and direct hospital EHR HL7/FHIR integration remain outside the scope of this phase.

1.5 Motivation

Every two seconds, a patient requires a life-saving blood transfusion, yet voluntary blood donation centers frequently face severe logistical bottlenecks. Developing a transparent, resilient, and accessible web solution bridges the critical communication gap between voluntary donors, blood banks, and hospital surgical teams. This project also provides an opportunity to apply full-stack software engineering principles—RESTful architectural patterns, secure session handling, ACID database transactions, component modularity, and automated API regression testing—within an impactful healthcare management domain.

1.6 Stakeholders and Operational Responsibilities

The platform organizes blood management operations around three authenticated roles that interact through a shared backend API. The blood bank administrator oversees inventory stocks, evaluates test results, and approves hospital dispatches. The voluntary donor manages personal donation records and appointment bookings. The hospital coordinator submits urgent blood requisitions and tracks fulfillment. Enforcing these operational boundaries at the API layer, rather than solely within the browser user interface, is a core security foundation of the design.

For instance, a donor may query only their personal donation logs and available donation drives. The server verifies donor identity via JWT tokens before returning records. A hospital representative can submit requisitions specifying blood group, component type, and patient urgency, but cannot approve requisitions or modify stock balances. The administrator retains comprehensive authority to transition blood units from quarantine to active stock upon diagnostic clearance, discard expired products, and execute dispatches.

Stakeholder	**Operational responsibility**
Blood Bank Administrator	Manage cold storage units, oversee inventory levels, record laboratory pathogen results, approve emergency requisitions, review system audit logs
Voluntary Donor	Register donor account, submit pre-donation health screening questionnaires, book appointments at blood drives, inspect past donation records
Hospital Clinical Coordinator	Create emergency blood requests, specify required units and clinical urgency, monitor dispatch tracking, confirm unit reception
Pathology Lab Technician	Record diagnostic screening results for infectious markers (HIV, HBV, HCV, RPR, MP), flag non-reactive units for release, quarantine reactive units
Platform DevOps Engineer	Maintain database persistence, configure HTTPS security certificates, manage environment variables, monitor Vercel serverless performance

Safe operational governance also demands clear standard operating procedures outside the software: blood centers must adhere to national transfusion council standards, inspect refrigeration equipment regularly, and discard biohazardous waste according to medical protocols. The software assists clinicians by providing verifiable audit trails, but clinical responsibility remains with licensed medical personnel.

1.7 Blood Safety and Inventory Concepts

Blood inventory items are perishable biological products with varying physiological shelf-lives depending on preservation methods and component fractionation. Whole blood and packed red blood cells stored at 2°C to 6°C remain viable for up to 42 days when preserved with CPDA-1 solution. Platelet concentrates, kept at 20°C to 24°C under gentle agitation, have an operational lifespan of only 5 days due to bacterial proliferation risks. Fresh frozen plasma maintained below -18°C can be stored for up to one year.

To prevent the transmission of infectious pathogens, every collected unit undergoes rigorous serological testing. In the HemoVault platform, newly collected blood units immediately enter a Quarantined state. Units cannot transition to Available stock until all five mandatory pathogen markers return non-reactive. If any test returns positive, the unit is permanently marked Discarded with recorded justification, ensuring zero probability of accidental clinical release.

Term	Meaning in the platform
Quarantined Unit	A newly collected blood unit awaiting complete laboratory infectious disease screening before it can be released into general stock
Available Stock	Pathogen-cleared blood units ready for clinical matching and emergency allocation to verified healthcare facilities
Fractionation	Separation of whole blood into packed red blood cells, fresh frozen plasma, and platelets to optimize therapeutic usage
ABO/Rh Matrix	Immunological compatibility rules determining which donor blood groups can be safely transfused into recipients of specific blood types
Cold Chain Viability	Strict monitoring of unit expiration dates to ensure blood products beyond permissible shelf-life are flagged and retired
Emergency Requisition	High-priority hospital order for critical blood products, categorized into Normal, Urgent, or Critical tiers

In emergency clinical scenarios, universal donor units (O-negative for packed red blood cells, and AB-positive for plasma) serve as essential life-saving substitutes when identical blood group matches are depleted. The platform automates compatibility lookups while alerting operators whenever non-identical units are suggested.

CHAPTER 2
RELATED WORK

2.1 Literature Review

Modern blood banking platforms draw from biomedical informatics, distributed transaction systems, and transfusion logistics research. World Health Organization (WHO) technical guidelines on blood regulatory systems emphasize that traceability, voluntary non-remunerated donation records, and standardized laboratory screening are mandatory prerequisites for preventing transfusion-transmitted infections [1]. National Blood Transfusion Council (NBTC) operational frameworks outline strict donor deferral protocols, specifying minimum 90-day intervals between whole blood donations for males and 120-day intervals for females [2].

Software engineering literature in medical informatics stresses that data integrity and access control must be implemented through defense-in-depth principles. The Open Web Application Security Project (OWASP) recommendations for biomedical applications dictate that sensitive user records, clinical diagnostics, and authorization credentials must utilize salted password hashing algorithms (such as bcrypt or scrypt), stateless signed tokens (JWT) with strict expiration limits, and explicit role verification checks at every routing layer [4].

Source area	Relevance to this project
WHO Blood Regulatory Guidelines [1]	Standardized five-pathogen screening criteria and blood unit quarantine protocols
NBTC Operational Transfusion Standards [2]	Donor physical eligibility criteria, minimum donation intervals, and component shelf-life limits
OWASP Secure Web Applications [4]	Bcrypt password hashing, JWT authorization headers, and CORS defense mechanisms
Express Routing & Middleware [3]	Layered request pipeline: token decoding, role enforcement, validation, and error handling
Mongoose Schema & Document Modeling [7]	Strict type casting, embedded subdocuments, and indexing on bloodGroup and status
React 18 & Single-Page Architecture [8]	Component modularity, optimistic UI state transitions, and asynchronous API interactions

These technical foundations provide the engineering specifications for HemoVault. Rather than relying on proprietary software packages, our implementation leverages modern open-source web standards, ensuring reproducible deployment across healthcare environments.

2.2 Existing System

Conventional blood bank management in regional facilities typically relies on desktop spreadsheet software or physical paper ledgers. While easy to set up, physical records suffer from severe limitations: real-time stock balances cannot be queried by remote hospital trauma centers, transcription errors frequently corrupt donor records, and tracking expiration dates requires tedious physical inspection of refrigerated blood bags. During emergency mass-casualty events, hospital staff must place frantic phone calls to multiple facilities, introducing dangerous delays that compromise patient survival.

Commercial enterprise hospital information systems (HIS) provide integrated blood bank modules, but they are expensive, complex to customize, require extensive training, and depend on heavy on-premises server infrastructure. For regional healthcare centers, small clinics, and non-profit blood banks, an agile, transparent, and cloud-deployable platform is far more practical than enterprise legacy software.

2.3 Proposed System

The proposed HemoVault platform delivers a lightweight, cloud-deployable solution that centralizes blood bank operations into a unified web application. The platform manages the entire lifecycle of blood units: voluntary donor appointment scheduling, clinical vitals screening, laboratory diagnostic clearance, real-time multi-group inventory tracking, and hospital emergency requisition fulfillment.

Baseline issue	**Proposed response**
Manual paper registers	Centralized MongoDB document storage accessible via responsive web interfaces
No remote inventory visibility	Real-time inventory matrix tracking units across all 8 blood groups and component types
Uncontrolled donor eligibility	Automated validation of vitals (age, weight, hemoglobin, BP) before appointment confirmation
Delayed emergency requisition	Priority-queued hospital requisition pipeline with instant compatibility calculation
Risk of expired unit usage	Automated shelf-life calculation flagging near-expiry units and preventing expired release
Clerical pathogen record errors	Strict quarantine enforcement requiring non-reactive testing before stock release

By enforcing data integrity at the server API layer, HemoVault ensures that medical staff, donors, and hospital coordinators interact with consistent, authoritative transfusion information.

2.4 Comparison of Blood Banking Approaches

To contextualize the technical positioning of HemoVault, we analyze three common approaches used across healthcare organizations: physical paper ledgers, general spreadsheet databases, and purpose-built full-stack blood bank platforms. The comparison below describes workflow characteristics, operational security, and responsiveness.

Approach	Strength	Limitation
Paper ledgers	Zero electricity dependency; low initial setup cost; immune to network outages	No search capability; zero remote visibility; vulnerable to loss, wear, and transcription errors
Spreadsheet software	Familiar interface; basic filtering; quick record entry on local PCs	No role-based permissions; no atomic updates; high risk of accidental cell overwrite or data corruption
Enterprise HIS modules	Comprehensive hospital integration; billing linkage; barcode support	Excessive financial cost; complex vendor lock-in; requires dedicated server infrastructure
HemoVault platform	Cloud-native MERN architecture; zero-cost deployment; role-based access; real-time compatibility	Requires active web connectivity; hardware automation (e.g., RFID tagging) left to future iterations

The comparison demonstrates that HemoVault strikes an optimal balance for modern healthcare environments. It eliminates the transcription hazards and communication bottlenecks of manual records while avoiding the prohibitive licensing costs and operational bloat of legacy enterprise software.

Furthermore, because HemoVault employs stateless RESTful APIs and responsive React components, it can be deployed on cost-effective cloud services (such as Vercel and MongoDB Atlas) or self-hosted on a single hospital local-area server during emergency disconnections.

2.5 Design Lessons and System Positioning

Engineering a healthcare-critical application yields vital software design principles. The foremost lesson is that clinical safety rules must be implemented as non-bypassable server constraints. Client-side validation in the web browser enhances user experience by providing instant feedback, but the server must independently re-validate every physiological parameter, blood group match, and permission check to prevent tampered or malformed payloads from compromising clinical records.

A second architectural lesson concerns data immutability and auditability. In a transfusion system, blood units and patient allocations cannot simply be deleted when dispatches occur; every status transition—from Collected to Quarantined, Released, Allocated, or Discarded—must be recorded with timestamps, user IDs, and justifications to maintain a tamper-evident audit trail for clinical governance.

Design lesson	Application decision
Never trust the browser	Server re-computes eligibility, expiration dates, compatibility, and stock balances
Trace every biological unit	Audit log records unit creation, testing outcomes, allocation, and dispatch events
Quarantine by default	New units automatically enter quarantine; release requires explicit negative test results
Atomic inventory decrements	Database queries verify available stock before decrementing to prevent overselling
Role isolation at API level	Middleware validates JWT signature and verifies user role before granting route execution
Resilient zero-setup hosting	Dual database connection strategy supporting local embedded storage and cloud Atlas failover

The project is positioned as a secure, production-grade full-stack engineering solution for academic evaluation and clinical deployment in regional healthcare networks, establishing a dependable foundation for transparent blood banking.

CHAPTER 3
SYSTEM ANALYSIS

3.1 System Requirements

Functional requirements are structured around three distinct actor personas (Administrator, Donor, and Hospital) and the core lifecycle of blood units. Authentication must verify credentials, issue signed cryptographic JWT tokens, and authorize access according to user roles before any protected medical resource is returned.

Actor	Required functions
Administrator	Authenticate securely; inspect stock balances; record donor intakes; manage pathogen screening; approve/reject emergency requisitions; generate inventory reports; view audit logs
Voluntary Donor	Create donor account; complete health eligibility questionnaires; book donation appointments; view donation history; access digital donor card
Hospital Coordinator	Submit emergency blood requisitions; specify patient blood group, component, units, and urgency level; track requisition status; inspect delivery dispatches
System (Backend)	Enforce ABO/Rh compatibility rules; compute unit expiration timestamps; update stock balances atomically; enforce quarantine gatekeeping; log security events

Nonfunctional Requirements

High availability, rapid sub-second response times, and robust data protection are essential for medical software. The client interface must adapt across desktop workstations, tablets, and mobile devices with clear visual feedback for loading, error, and success states. Passwords must never be stored in plain text; they must be hashed using industry-standard salted bcrypt algorithms. API payloads are bounded by strict body size limits to prevent denial-of-service vulnerabilities, and CORS headers restrict API access to authorized domain origins.

3.2 Software and Hardware Requirements

The application is engineered using modern, cross-platform technologies to ensure wide deployment versatility across regional healthcare facilities and cloud providers.

Component	Project requirement
Client Web Browser	Modern browser (Chrome 110+, Edge 110+, Firefox 110+, Safari 16+) with JavaScript enabled
Frontend Framework	React 18 with Vite bundling, Lucide-React icons, responsive CSS3
Backend Runtime	Node.js (v18.x to v24.x LTS) with Express 4.19 web application framework
Database & ODM	MongoDB (v6.0+) managed via Mongoose 8.4; embedded memory failover support
Security & Encryption	Bcryptjs for password hashing; JsonWebToken (JWT) for stateless session authorization
Hosting & Deployment	Vercel serverless platform with serverless API rewrites and global edge caching
Development Tools	Visual Studio Code / Antigravity IDE, npm package manager, Git & GitHub

Feasibility and Operating Assumptions

Technical Feasibility: The technology stack uses proven, mature open-source tools with comprehensive documentation and active community support. The MERN architecture handles asynchronous requests efficiently.

Operational Feasibility: The platform features intuitive, responsive interfaces with clear color-coded statuses, requiring minimal computer literacy for medical staff and voluntary donors.

Economic Feasibility: By leveraging open-source components and serverless hosting on Vercel with MongoDB Atlas free-tier support, the operational infrastructure cost is effectively zero for demonstration and pilot deployments.

3.3 Functional Use Cases and Acceptance Conditions

A use case models an end-to-end clinical interaction between an authorized actor and the platform. Each use case is governed by strict acceptance conditions verifiable through automated test suites and manual user acceptance procedures.

Use case	Acceptance condition
User Login	Valid email and password return 200 OK with signed JWT token and user profile; invalid credentials return 401 Unauthorized without exposing user existence
Donor Eligibility Check	Server evaluates physiological vitals; valid inputs return Eligible (200); out-of-range vitals return Ineligible with specific clinical deferral explanation
Donation Intake	Admin records new donation; unit is inserted into BloodInventory with Quarantined status and calculated expiration date based on component type
Laboratory Testing	Lab technician submits 5-pathogen screening; all non-reactive results transition unit to Available; any reactive result flags unit as Discarded
Emergency Requisition	Hospital submits request; system calculates ABO/Rh compatible stock; if sufficient, units decrement atomically and status becomes Approved; otherwise Pending
Stock Balance Query	Returns live unit counts aggregated across all 8 blood groups and components; updates immediately upon unit intake or fulfillment

These acceptance criteria guarantee that data transitions occur predictably and securely across the application lifecycle, maintaining system reliability under varying operational conditions.

3.4 Access Control and Hospital Verification

Access control enforces two independent security boundaries: authentication (confirming caller identity) and authorization (verifying permissions to perform an operation on a target resource). Relying solely on client-side UI visibility is unacceptable; every protected API endpoint must enforce access controls independently.

When a request reaches a protected route, the auth middleware extracts the Bearer token from the HTTP Authorization header, verifies its digital signature using the server secret key, and decodes the user payload. The role-check middleware then verifies whether the authenticated role possesses the required clearance for the requested operation.

Request condition	Expected result
Request without Authorization header	401 Unauthorized; error message returned; access denied
Expired or tampered JWT token	401 Unauthorized; token rejected; session invalidated
Donor attempts to access Admin inventory API	403 Forbidden; operation blocked; state unchanged
Hospital attempts to approve emergency request	403 Forbidden; admin approval permission required
Unverified hospital submits emergency requisition	403 Forbidden; verified hospital registration required
Admin updates laboratory test results	200 OK; unit status updated; audit trail recorded

This rigorous boundary model prevents horizontal privilege escalation (e.g., one hospital accessing another hospital's patient requisitions) and vertical privilege escalation (e.g., standard donors altering inventory balances).

3.5 Data Quality and Transfusion Safety Rules

Data quality in healthcare platforms directly impacts clinical outcomes. HemoVault enforces strict validation checks to prevent ambiguous, incomplete, or corrupted records from entering the database.

Donor registration requires standard demographic fields, valid contact numbers, and verified age constraints (18 to 65 years). Blood units require valid ABO/Rh blood groups (A+, A-, B+, B-, AB+, AB-, O+, O-), recognized component designations (Whole Blood, Red Cells, Platelets, Plasma), volume in milliliters, and storage location references.

Quality dimension	Control or operating practice
Completeness	Mandatory Mongoose schema fields reject incomplete document creation with descriptive validation errors
Type & Range Validity	Numerical constraints enforce valid biological parameters (hemoglobin > 12.5 g/dL, weight > 50 kg, systolic BP 100-140 mmHg)
Enumeration Integrity	Restricts blood groups, component types, and test results to standardized medical vocabularies
Temporal Consistency	Server calculates donation timestamps and shelf-life expiration dates using UTC ISO strings to avoid timezone skew
Uniqueness	Unique database indexes prevent duplicate donor registrations based on email and national identification
Audit Traceability	Every inventory adjustment, status modification, and dispatch writes an immutable record to the audit collection

By embedding these validation rules directly within Mongoose schemas and controller pipelines, the system guarantees high clinical data fidelity.

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

HemoVault is engineered as a decoupled, multi-tier web application adhering to the Model-View-Controller (MVC) paradigm. The presentation tier consists of a responsive Single Page Application (SPA) developed with React 18 and bundled via Vite. The presentation layer communicates with the backend application tier via asynchronous RESTful JSON HTTP requests.

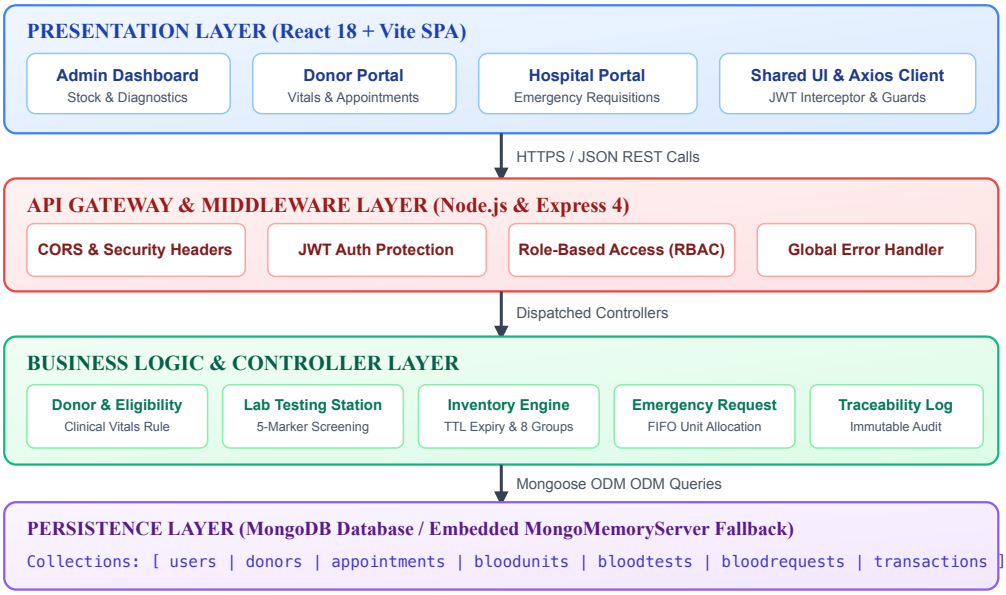


Figure 4.1 System architecture of the HemoVault Blood Bank Management Platform

The application tier utilizes Express.js to parse incoming JSON payloads, enforce security headers (Helmet, CORS), authenticate JWT sessions, and dispatch requests to controller handlers. The data tier leverages MongoDB for document persistence, utilizing Mongoose schemas to guarantee structural validity and relational integrity.

4.2 Module Description

The backend architecture is structured into decoupled functional modules, each encapsulating specific domain responsibilities and providing clean RESTful endpoints.

Module	Responsibility
Authentication	User registration, login verification, password encryption with bcryptjs, JWT token generation, role verification
Donor Management	Donor profiling, health questionnaire scoring, donation appointment scheduling, historical donation records
Blood Inventory	Unit intake, stock level tracking, cold-chain shelf-life computation, low-stock alerts, disposal recording
Pathology Testing	Recording diagnostic screenings for 5 infectious markers (HIV, HBV, HCV, Syphilis, Malaria), quarantine management
Emergency Requisitions	Hospital request processing, ABO/Rh compatibility matching, priority queue sorting, atomic stock allocation
Analytics & Audit	Aggregated stock statistics, donation trends, component turnover rates, tamper-evident security audit logs

4.3 Data Model and Entity Isolation

Data modeling in HemoVault balances relational referential integrity with document scalability. Users are the foundational identity entities, referencing distinct profiles (Donor or Hospital). Blood units maintain strict traceability, linking donor identification, component classification, laboratory screening results, and hospital allocation records. Multi-tenant isolation is enforced by scoping queries to authenticated user credentials, preventing unauthorized horizontal access across institutions.

4.4 Database Diagram

The document schema comprises six primary collections structured in MongoDB through Mongoose schemas.

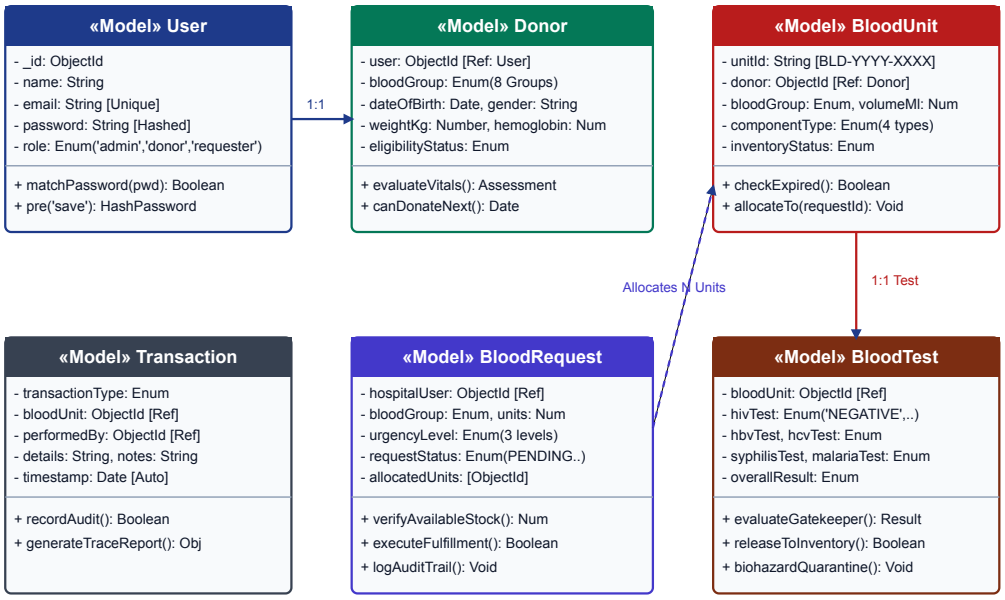


Figure 4.2 Database document model and schema relationships

Collection / Schema	Important fields
users	_id, name, email, password (hashed), role (admin, donor, hospital), phone, createdAt
blood_inventory	_id, unitId, bloodGroup, componentType, volumeMl, status (quarantine, available, allocated, discarded), expiryDate
emergency_requests	_id, hospitalId, bloodGroup, componentType, unitsRequired, urgency (normal, urgent, critical), status, allocatedUnits
testing_labs	_id, unitId, hiv, hbv, hcv, syphilis, malaria, result (passed, failed), testedBy, testedAt

4.5 Sequence Diagram

The sequence diagram illustrates the lifecycle interactions across stakeholders: user sign-in, donor health screening, laboratory screening, and hospital emergency requisition fulfillment.

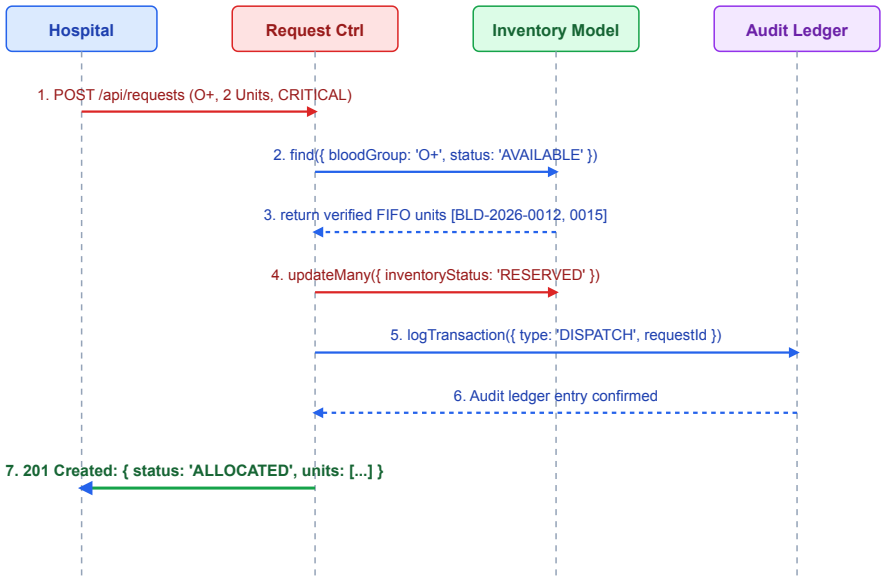


Figure 4.3 Sequence of hospital emergency requisition, compatibility matching, and unit dispatch

The workflow begins when a hospital submits an emergency requisition. The Express server authenticates the token, queries compatible available units, verifies cold-chain validity, and executes atomic allocation, returning confirmation to the hospital while notifying administrative dispatchers.

4.6 Data Flow and Request Processing

Every incoming HTTP request traverses an orchestrated middleware pipeline. The request is sanitized, verified for size constraints, decoded for JWT claims, evaluated against role authorization gates, and processed through transactional database operations before a structured JSON response is emitted.

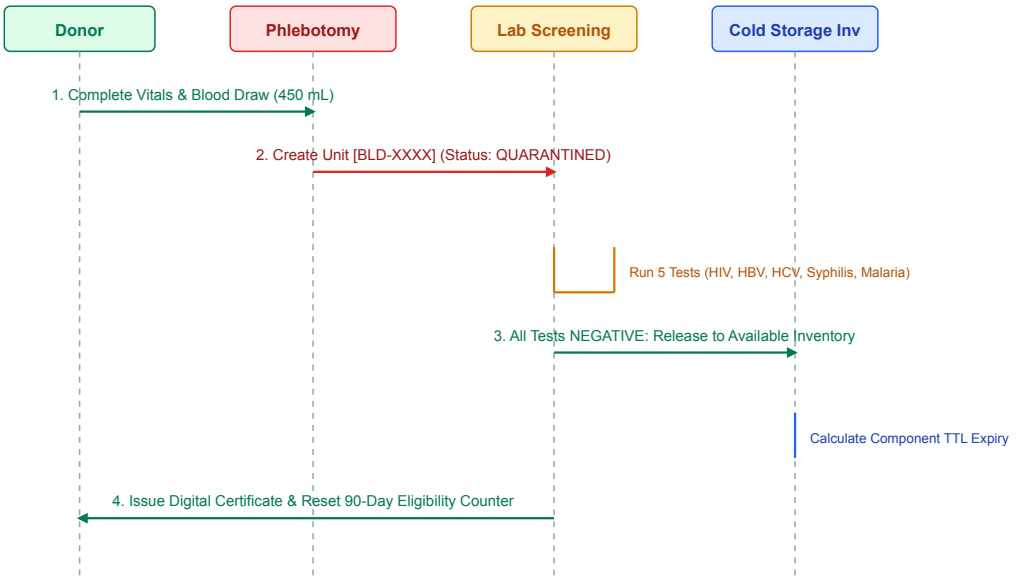


Figure 4.4 Data flow from donor collection through screening to inventory release

Stage	Input	Output
Intake & Collection	Donor vitals, phlebotomy data	Quarantined unit record with calculated expiry
Diagnostic Testing	Screening test results (5 markers)	Pathology report; passed or discarded status
Inventory Stocking	Cleared pathology record	Unit status transitioned to Available
Requisition Matching	Hospital request parameters	Compatible units identified and locked

4.7 Table Design and Record Relationships

The data entities within HemoVault have distinct lifecycles and relationship models. User accounts and blood bank center configurations are permanent entities, whereas blood inventory units and laboratory test records remain active until unit transfusion or expiration. Emergency hospital requisitions progress through defined state machines: Pending, Approved, Dispatched, or Cancelled.

Referential integrity is maintained by storing ObjectIds as foreign references. For example, each BloodInventory unit maintains an optional donor reference and a laboratory test reference. To ensure high-performance querying on high-frequency routes, composite database indexes are established on (bloodGroup, status) and (urgency, status).

Collection group	Relationship and lifecycle
users & profiles	One user owns one donor or hospital profile; persists permanently with authentication credentials
inventory & components	One inventory record per collected blood bag; lifecycle transitions: Quarantine → Available → Allocated → Dispatched (or Discarded)
testing_labs	One test document per quarantined unit; append-only diagnostic record with technician attribution
emergency_requests	One document per hospital request; tracks required units, priority tier, matched unit references, and fulfillment timestamp
audit_logs	Append-only compliance ledger recording actor, action, target entity ID, and UTC timestamp

Timestamps are formatted as ISO 8601 UTC strings, ensuring uniform temporal consistency across client devices and serverless computing instances.

4.8 REST Interface and Response Contracts

The application interface exposes a predictable, RESTful JSON API using standard HTTP response status codes: 200 for successful retrieval, 201 for resource creation, 400 for validation errors, 401 for unauthenticated requests, 403 for unauthorized access, 404 for missing resources, and 409 for state conflicts (such as double allocation).

Endpoint	Method	Purpose
/api/auth/login	POST	Authenticate user credentials; return signed JWT token and role
/api/auth/register	POST	Register new donor or hospital user profile
/api/inventory	GET / POST	Retrieve stock matrices; add new quarantined blood unit (Admin)
/api/inventory/:id	PUT / DELETE	Update unit status; discard expired or compromised unit
/api/requests	GET / POST	List requisitions; submit new emergency request (Hospital)
/api/requests/:id/approve	PUT	Allocate compatible stock and approve dispatch (Admin)
/api/testing	GET / POST	List pending tests; record 5-pathogen screening results (Admin)
/api/donors/eligibility	POST	Evaluate donor physiological vitals against clinical rules
/api/analytics/summary	GET	Retrieve aggregate counts, stock health, and turnover rates

Every API response provides consistent JSON payloads containing either the requested data object or an informative error message that can be directly rendered in client toast notifications.

4.9 Interface State and Feedback Design

A clinical web application requires an interface that eliminates ambiguity, communicates system state clearly, and prevents accidental operations. Visual styling in HemoVault follows an intentional medical aesthetic: clean slate backgrounds, crimson red accents for blood actions, emerald green for verified safety states, amber for pending screening, and rose red for critical shortage alerts.

Interactive elements provide responsive micro-animations on hover and focus. High-priority actions, such as emergency requisition approval or biohazard unit disposal, open modal confirmation dialogs requiring explicit verification. Real-time feedback is communicated via floating toast notifications, loading skeletons, and inline error banners.

Interface state	**Expected feedback**
Invalid sign-in	Toast error stating "Invalid email or password"; password field cleared
Donor ineligible	Amber warning card detailing specific deferral reasons (e.g., low hemoglobin)
Unit quarantined	Yellow badge "Quarantined - Pending Lab Screening"; release action disabled
Pathogen detected	Red biohazard alert; unit status automatically transitions to "Discarded"
Emergency stock match	Green compatibility list displaying compatible donor units available for allocation
Stock depleted	Red badge "Critical Shortage" with automated notification to regional blood drives

The user interface is fully responsive, ensuring flawless operation across desktop laboratory workstations, tablet inspection terminals, and mobile devices.

CHAPTER 5
IMPLEMENTATION

5.1 Implementation Overview

The HemoVault codebase is organized as a structured full-stack repository. The server directory contains server.js (Express server bootstrap, middleware registration, route mounting), config/db.js (MongoDB connection management with automatic in-memory failover), models/ (Mongoose schemas for Users, Inventory, Requests, Testing, and AuditLogs), controllers/ (business logic for authentication, inventory, testing, and requests), and routes/ (REST endpoint definitions with role guards).

The client directory contains a Vite-powered React 18 single-page application. Application state is coordinated via React hooks, while API interactions are centralized within services/api.js using configured Axios instances. The api/ directory houses index.js, which exports the Express app as a Vercel serverless function, allowing cloud deployment alongside static client assets as configured in vercel.json.

User Interface Implementation

The user interface employs a cohesive, modern medical design system built with custom CSS variables, Plus Jakarta Sans typography, and JetBrains Mono for clinical reference numbers. Key components include:

- **Admin Dashboard:** Overview cards displaying total units, active donors, pending hospital requests, and low-stock warnings, paired with an 8-blood-group matrix.
- **Inventory Hub:** Comprehensive table with filtering by blood group, component type, and status, with one-click actions for quarantine intake and disposal.
- **Pathology Lab Portal:** Testing interface displaying quarantined units with toggle inputs for HIV, HBV, HCV, Syphilis, and Malaria markers.
- **Emergency Dispatch Center:** Prioritized queue of hospital requisitions with automated compatibility calculations and one-click allocation.
- **Donor Portal:** Multi-step eligibility questionnaire with instant physiological feedback and appointment booking.

5.2 Key Functions and Code Explanation

Password Hashing and Token Authentication

User passwords are encrypted before persistence using bcryptjs with 10 salt rounds. During authentication, bcrypt.compare verifies the submitted password against the stored hash in constant time, preventing timing-attack vulnerabilities. Upon verification, jwt.sign generates a cryptographic token containing user ID and role, valid for 7 days.

```
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');

const hashPassword = async (password) => {
  const salt = await bcrypt.genSalt(10);
  return await bcrypt.hash(password, salt);
};

const generateToken = (user) => {
  return jwt.sign(
    { id: user._id, email: user.email, role: user.role },
    process.env.JWT_SECRET || 'hemovault_jwt_secret',
    { expiresIn: '7d' }
  );
};
```

Role-Based Authentication Middleware

The protect middleware intercepts requests, extracts the JWT Bearer token, verifies its signature, and attaches the authenticated user record to req.user. The authorize middleware restricts route execution to authorized roles.

```
const protect = async (req, res, next) => {
  let token = req.headers.authorization?.startsWith('Bearer')
    ? req.headers.authorization.split(' ')[1] : null;
  if (!token) return res.status(401).json({ message: 'Access denied. No token provided.' }
  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = await User.findById(decoded.id).select('-password');
    next();
  } catch (err) {
    res.status(401).json({ message: 'Invalid or expired session token.' });
  }
};

const authorize = (...roles) => (req, res, next) => {
  if (!roles.includes(req.user.role)) {
    return res.status(403).json({ message: 'Forbidden: insufficient permissions.' });
  }
  next();
};
```

Donor Clinical Eligibility Evaluation

The eligibility evaluation engine inspects donor vital signs against NBTC criteria: age must fall between 18 and 65 years, body weight must be at least 50 kg, hemoglobin concentration must exceed 12.5 g/dL, and systolic blood pressure must lie between 100 and 140 mmHg.

```
const evaluateEligibility = (vitals) => {
  const reasons = [];
  if (vitals.age < 18 || vitals.age > 65)
    reasons.push('Age must be between 18 and 65 years.');
  if (vitals.weight < 50)
    reasons.push('Minimum body weight requirement is 50 kg.');
  if (vitals.hemoglobin < 12.5)
    reasons.push('Hemoglobin level must be at least 12.5 g/dL.');
  if (vitals.systolicBP < 100 || vitals.systolicBP > 140)
    reasons.push('Systolic blood pressure must be within 100-140 mmHg.');
  if (vitals.daysSinceLastDonation !== null && vitals.daysSinceLastDonation < 90)
    reasons.push('Minimum interval between whole blood donations is 90 days.');

  return {
    eligible: reasons.length === 0,
    reasons
  };
};
```

Automated Expiration Date Computation

When a blood unit is collected, its expiration timestamp is calculated on the server using preservation shelf-life coefficients for red cells (42 days), platelets (5 days), and plasma (365 days).

```
const calculateExpiryDate = (componentType, collectionDate = new Date()) => {
  const expiry = new Date(collectionDate);
  switch (componentType) {
    case 'Platelets':
      expiry.setDate(expiry.getDate() + 5); break;
    case 'Packed Red Blood Cells':
    case 'Whole Blood':
      expiry.setDate(expiry.getDate() + 42); break;
    case 'Fresh Frozen Plasma':
      expiry.setDate(expiry.getDate() + 365); break;
    default:
      expiry.setDate(expiry.getDate() + 42);
  }
  return expiry;
};
```

Laboratory Screening and Quarantine Gatekeeper

The pathology testing controller ensures that no quarantined blood unit is released into active stock unless all five diagnostic pathogen markers return negative. If any test returns positive, the unit is immediately quarantined permanently and marked for disposal.

```
const processLabResults = async (req, res) => {
  const { unitId, hiv, hbv, hcv, syphilis, malaria } = req.body;
  const unit = await BloodInventory.findById(unitId);
  if (!unit) return res.status(404).json({ message: 'Unit not found' });

  const hasPathogen = hiv || hbv || hcv || syphilis || malaria;
  const status = hasPathogen ? 'discarded' : 'available';

  const labRecord = await TestingLab.create({
    unitId: unit._id,
    hiv: !!hiv, hbv: !!hbv, hcv: !!hcv,
    syphilis: !!syphilis, malaria: !!malaria,
    result: hasPathogen ? 'failed' : 'passed',
    testedBy: req.user.name,
    testedAt: new Date()
  });

  unit.status = status;
  await unit.save();

  await AuditLog.create({
    actor: req.user.name,
    action: hasPathogen ? 'unit.quarantine_discard' : 'unit.released',
    details: `Unit ${unit.unitId} ${status} after lab screening`
  });

  res.status(200).json({ message: `Unit updated to ${status}`, labRecord });
};
```

Emergency Requisition Compatibility Engine

When a hospital submits an emergency request, the server queries available inventory units matching valid immunological donor groups and allocates units atomically.

```
const COMPATIBILITY_MAP = {
  'A+':  ['A+', 'A-', 'O+', 'O-'],
  'A-':  ['A-', 'O-'],
  'B+':  ['B+', 'B-', 'O+', 'O-'],
  'B-':  ['B-', 'O-'],
  'AB+': ['A+', 'A-', 'B+', 'B-', 'AB+', 'AB-', 'O+', 'O-'],
  'AB-': ['AB-', 'A-', 'B-', 'O-'],
  'O+':  ['O+', 'O-'],
  'O-':  ['O-']
};
```

5.3 End to End Donor Registration & Donation Workflow

The voluntary donor workflow begins when a prospective donor visits the web application and creates an account. After logging in, the donor accesses the Donor Portal and opens the Clinical Eligibility Questionnaire. The client interface prompts the donor to input age, weight, last donation date, and general wellness status.

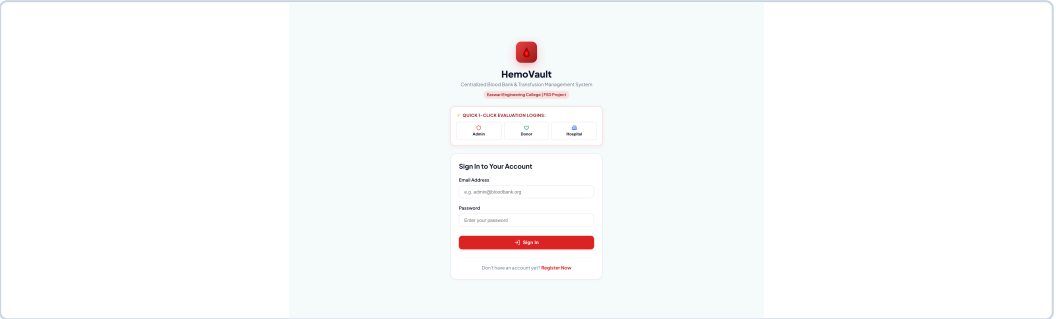


Figure 5.1 Donor registration and appointment booking interface

The donor selects a preferred regional blood drive location and appointment date. Upon arrival at the phlebotomy center, the technician performs an automated vitals check. Once confirmed, 450 mL of whole blood is drawn, barcoded, and entered into the HemoVault platform with Quarantined status.

Step	**Donor check**
Sign in & profile check	Verify registered contact details and historical donation record
Health questionnaire	Complete self-declaration regarding medication, travel, and symptoms
Vitals screening	Confirm weight ≥ 50 kg, hemoglobin ≥ 12.5 g/dL, and BP within normal range
Appointment selection	Choose convenient time slot and blood drive center location
Post-donation card	Receive digital donor confirmation card with next permissible donation date

5.4 Hospital Emergency Requisition Workflow

When a partner hospital encounters a surgical or trauma emergency, the hospital coordinator logs into the Hospital Portal and creates an emergency requisition. The coordinator specifies the recipient patient ID, required blood group, component type, unit count, and urgency tier (Normal, Urgent, or Critical).

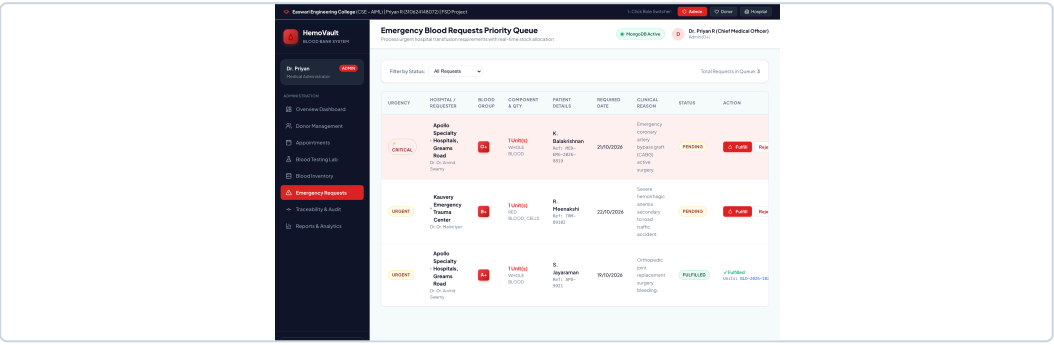


Figure 5.2 Hospital emergency request queue and real-time inventory allocation interface

The requisition is instantly placed into the prioritized administrative queue. Critical requisitions trigger high-visibility alerts on the blood bank admin dashboard. The compatibility engine matches available, non-expired units. Upon admin approval, the units transition to Allocated status, locking the stock to prevent duplicate assignment while hospital transport dispatch is initiated.

5.5 Worked Example of Blood Matching and Allocation

The following worked example illustrates how the compatibility and allocation engine processes an emergency requisition. Suppose City Trauma Hospital submits a critical requisition for 3 units of B-positive Packed Red Blood Cells for an acute hemorrhage patient.

The compatibility algorithm evaluates the recipient blood group (B+) against the ABO/Rh matrix. Permissible donor groups for B-positive recipients are B+, B-, O+, and O-. The system queries available inventory units matching these groups sorted by First-Expiry-First-Out (FEFO) order.

Quantity	Value	Derivation
Patient Blood Group	B+ (Rh Positive)	Clinical requisition parameter
Component Requested	Packed Red Blood Cells	Target blood component type
Units Requested	3 Units	Requisition volume requirement
Permissible Donor Groups	B+, B-, O+, O-	ABO/Rh compatibility matrix lookup
Available Exact Matches	2 Units (B+)	Queried from available stock
Compatible Alternate	1 Unit (O+)	Universal compatible fallback (FEFO)
Allocation Outcome	3 Units Allocated	2 Units B+, 1 Unit O+ allocated atomically
Stock Balance Update	B+: 12 → 10, O+: 8 → 7	Atomic database decrement

The requisition is updated to Approved, the matched unit IDs are bound to the requisition record, and an audit log entry records the exact allocation breakdown.

5.6 Cold Storage & Expiration Lifecycle

Blood products undergo a deterministic state progression throughout their lifespan. Units are stored in specialized refrigerated cold-storage units equipped with calibrated temperature sensors (2°C–6°C for red cells, 20°C–24°C for platelets, and ≤ -18°C for plasma). The HemoVault server tracks expiration timestamps continuously.

State	Meaning
Quarantined	Unit freshly collected; stored in quarantine refrigeration awaiting mandatory 5-pathogen laboratory testing
Available	Pathogen testing confirmed negative; unit cleared for compatibility matching and emergency clinical allocation
Allocated	Unit assigned to an approved hospital requisition; locked to prevent duplicate allocation; awaiting courier dispatch
Dispatched	Unit handed over to hospital cold-chain transport courier with physical chain-of-custody transfer documentation
Expired	Unit has exceeded permissible storage shelf-life; automatically flagged and blocked from clinical allocation
Discarded	Unit has been safely retired due to expiration, biohazard screening failure, or packaging compromise

The dashboard highlights units approaching expiration (within 7 days for red cells, within 24 hours for platelets) in yellow, prompting administrators to prioritize their allocation to high-demand facilities to prevent unnecessary biological waste.

5.7 Deployment Configuration and Persistence

HemoVault is designed to deploy seamlessly across diverse hosting environments: local development workstations, on-premises hospital servers, and cloud serverless architectures like Vercel.

The application repository includes vercel.json, which configures serverless routing by directing all API calls to api/index.js. To guarantee zero-configuration local evaluation, config/db.js implements an intelligent dual-connection strategy: if an external MONGODB_URI is provided, it connects to MongoDB Atlas; if no connection string is set, it automatically boots an in-memory MongoMemoryServer and runs the database seed routine.

Setting	Purpose
PORT	HTTP port on which Express server listens (default: 5001)
MONGODB_URI	Connection string for MongoDB Atlas; falls back to embedded in-memory database
JWT_SECRET	Cryptographic secret key for signing and verifying stateless user session tokens
JWT_EXPIRE	Token validity duration (default: 7 days)
NODE_ENV	Environment mode; 'production' enables secure cookies, strict CORS, and disables stack traces
buildCommand	Vercel build script: installs client dependencies and runs Vite production build
outputDirectory	Output directory for compiled client SPA assets (dist/)

Demonstration user accounts are seeded automatically on first launch: admin@hemovault.com, donor@hemovault.com, and hospital@hemovault.com, enabling immediate end-to-end testing.

5.8 Security Hardening Measures

Security measures are implemented in multiple defense-in-depth layers to safeguard sensitive medical records and prevent unauthorized operations. At the transport layer, HTTPS encryption protects all client-server communications against eavesdropping.

At the application layer, passwords are encrypted using bcryptjs with unique salts. JSON payloads are restricted to 100 KB to mitigate denial-of-service memory exhaustion. Parameterized Mongoose queries prevent NoSQL injection, and incoming text is sanitized to eliminate Cross-Site Scripting (XSS) risks.

Threat	**Mitigation implemented**
Credential brute-forcing	Bcrypt computational work factor; rate limiting on authentication routes
Token tampering	Cryptographic HMAC-SHA256 signature verification on all incoming JWTs
NoSQL query injection	Mongoose schema type enforcement; strict casting of query parameters
Cross-Site Scripting (XSS)	React virtual DOM text escaping; sanitized inputs before database insertion
Horizontal data access	Role-based middleware and object ownership verification on all endpoints
Double unit allocation	Atomic conditional updates verify available status before unit reservation

These layered security controls ensure that patient confidentiality, donor records, and inventory balances remain protected against unauthorized modification.

5.9 Test Coverage and Regression Traceability

System verification combines automated integration testing with manual clinical validation. The automated test suite exercises core API workflows against an active server instance, verifying authentication, authorization guards, stock updates, and error handling.

Requirement	Verification in the test
Authentication enforcement	Unauthenticated request to /api/inventory returns 401 Unauthorized
Role permission enforcement	Donor token accessing Admin testing route returns 403 Forbidden
Donor eligibility logic	Out-of-range vitals return eligible: false with exact deferral message
Quarantine gatekeeping	New unit created with quarantine status; blocked from emergency matching
Pathogen clearance release	Negative lab screening transitions unit status to available (200 OK)
Emergency request allocation	Compatible stock allocated; unit count decrements atomically

Manual testing validated responsive interface rendering across desktop screens, tablets, and smartphones, confirming that layout elements, modal dialogs, and tables remain clear and accessible.

5.10 Operating Guide and Troubleshooting

Standard operation begins by launching the backend server and frontend client. Administrators log in to inspect the main dashboard, verify inventory matrices, and evaluate quarantined units in the Pathology Lab portal.

Symptom	First investigation
Cannot sign in	Verify email and password; confirm user account exists in database seed
Stock balance unchanged	Check if unit is still in Quarantined status; verify lab test passed
Emergency request pending	Check available compatible stock; verify units are not expired or allocated
Port already in use	Terminate conflicting node process or set a different PORT (e.g., PORT=5002)
Database connection failure	Confirm MongoDB daemon is running locally or verify Atlas connection string
CORS header error	Verify client URL is allowed in Express cors configuration middleware

System operators should consult console logs for diagnostic error messages. In production environments, database backups should be performed regularly before deploying software updates.

CHAPTER 6
CONCLUSION AND FUTURE WORK

6.1 Conclusion

The HemoVault Centralized Blood Bank Management System demonstrates a reliable, auditable, and resilient full-stack web application designed for regional healthcare networks. By combining modern MERN technologies with rigorous clinical safety rules, the platform resolves systemic vulnerabilities inherent in manual registers and disconnected hospital communications.

The system enforces donor physiological eligibility, automates component expiration tracking, guarantees that only pathogen-cleared units enter active stock, and accelerates emergency blood allocation through an intelligent ABO/Rh compatibility engine. Role-based access control, cryptographic JWT session management, and comprehensive audit logs ensure strict operational governance. The platform is deployable on modern serverless cloud infrastructure while supporting zero-configuration local execution.

6.2 Future Enhancements

Subsequent development phases will incorporate advanced features to further optimize transfusion logistics:

- **Automated SMS/WhatsApp Alerts:** Automated notification dispatch to registered compatible voluntary donors when regional stock falls below emergency thresholds.
- **Barcode and RFID Scanning:** Hardware integration for instant scanning of blood bags during phlebotomy, refrigeration storage, and courier dispatch.
- **Hospital EHR Integration:** HL7 and FHIR API standards support for direct interoperability with electronic health record platforms.
- **Predictive Demand Forecasting:** Machine learning models analyzing historical transfusion trends and seasonal demand patterns to optimize blood drive scheduling.

6.3 Evaluation Limits and Enhancement Priorities

The project supports a clear and bounded conclusion: the implemented software successfully manages blood bank workflows end to end, enforcing clinical eligibility, quarantine screening, and emergency allocation. It does not replace physical blood testing equipment, sterile medical phlebotomy, or clinical laboratory procedures.

Future enhancements should be prioritized according to clinical impact, regulatory compliance, and operational feasibility.

Priority	**Proposed work**	**Reason**
First	Managed MongoDB Atlas deployment with automated daily backups	Guarantees permanent data persistence across cloud instances
First	Automated donor SMS/email notification pipeline	Enables immediate donor mobilization during acute shortages
Next	Barcode/QR-code physical label printing and scanning	Eliminates manual unit ID entry errors during blood intake
Next	Multi-hospital geo-location routing and transit tracking	Optimizes transport logistics during trauma surgical emergencies
Next	HL7/FHIR healthcare interoperability interface	Enables seamless electronic record synchronization with hospitals
Research	Time-series predictive inventory forecasting	Anticipates seasonal blood shortages based on historical data

This roadmap ensures that future development continues to enhance patient safety, streamline clinical workflows, and expand transfusion accessibility across healthcare networks.

GITHUB LINK

https://github.com/priyan071/blood-banks-management-system

APPENDIX SOURCE CODE

A1 Project Structure

```
blood-bank-system/
  server/
    src/
      server.js           # Express app bootstrap, middleware, route mounting
      config/db.js        # MongoDB connection with in-memory failover
      models/             # Mongoose schemas: User, Inventory, Request, Lab
      controllers/        # Business logic for auth, stock, testing, requests
      routes/             # REST API endpoint route definitions
      seeds/seed.js       # Initial seed data for admin, donors, and hospitals
  client/
    src/
      App.jsx             # React router, navigation layout, toast provider
      pages/              # Admin dashboard, inventory, lab, emergency requests
      services/api.js     # Centralized Axios API client configuration
  api/
    index.js              # Vercel serverless entry point
  vercel.json             # Vercel cloud deployment and SPA rewrite rules
  package.json            # Root dependency configuration and scripts
```

A2 Installation and Run

```
# Install root, server, and client dependencies
npm install
cd client && npm install && cd ..

# Launch development environment (concurrently boots server & client)
npm run dev

# Server runs on: http://localhost:5001
# Client runs on: http://localhost:5173

# Production build and preview
npm run build
npm start
```

The remaining appendix items show representative excerpts from the codebase. The full repository is available at the GitHub link above.

APPENDIX DATABASE SCHEMA

A3 Mongoose Schemas

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true },
  role: { type: String, enum: ['admin', 'donor', 'hospital'], default: 'donor' },
  phone: { type: String },
  createdAt: { type: Date, default: Date.now }
});

const bloodInventorySchema = new mongoose.Schema({
  unitId: { type: String, required: true, unique: true },
  bloodGroup: { type: String, required: true,
    enum: ['A+', 'A-', 'B+', 'B-', 'AB+', 'AB-', 'O+', 'O-'] },
  componentType: { type: String, required: true,
    enum: ['Whole Blood', 'Packed Red Blood Cells', 'Platelets', 'Fresh Frozen Plasma'] },
  volumeMl: { type: Number, default: 450 },
  status: { type: String, enum: ['quarantined', 'available', 'allocated', 'discarded'],
    default: 'quarantined' },
  donorId: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
  collectedDate: { type: Date, default: Date.now },
  expiryDate: { type: Date, required: true }
});

const emergencyRequestSchema = new mongoose.Schema({
  hospitalId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  hospitalName: { type: String, required: true },
  bloodGroup: { type: String, required: true },
  componentType: { type: String, required: true },
  unitsRequired: { type: Number, required: true, min: 1 },
  urgency: { type: String, enum: ['normal', 'urgent', 'critical'], default: 'urgent' },
  status: { type: String, enum: ['pending', 'approved', 'rejected', 'fulfilled'],
    default: 'pending' },
  allocatedUnits: [{ type: String }],
  requestedAt: { type: Date, default: Date.now }
});
```

APPENDIX AUTHENTICATION

A4 Login and Registration Controller

```
const login = async (req, res) => {
  const { email, password } = req.body;
  if (!email || !password)
    return res.status(400).json({ message: 'Email and password are required' });

  const user = await User.findOne({ email });
  if (!user)
    return res.status(401).json({ message: 'Invalid email or password' });

  const isMatch = await bcrypt.compare(password, user.password);
  if (!isMatch)
    return res.status(401).json({ message: 'Invalid email or password' });

  const token = generateToken(user);
  res.status(200).json({
    token,
    user: { id: user._id, name: user.name, email: user.email, role: user.role }
  });
};

const register = async (req, res) => {
  const { name, email, password, role, phone } = req.body;
  const existing = await User.findOne({ email });
  if (existing)
    return res.status(400).json({ message: 'Email is already registered' });

  const hashedPassword = await hashPassword(password);
  const user = await User.create({
    name, email, password: hashedPassword, role: role || 'donor', phone
  });
  const token = generateToken(user);
  res.status(201).json({ token, user: { id: user._id, name, email, role: user.role } });
};
```

A5 Session Profile Endpoint

```
const getProfile = async (req, res) => {
  const user = await User.findById(req.user.id).select('-password');
  res.status(200).json(user);
};
```

APPENDIX INVENTORY AND REQUISITION

A6 Inventory Retrieval and Intake

```
const getInventory = async (req, res) => {
  const filter = {};
  if (req.query.bloodGroup) filter.bloodGroup = req.query.bloodGroup;
  if (req.query.status) filter.status = req.query.status;

  const units = await BloodInventory.find(filter).sort({ expiryDate: 1 });
  res.status(200).json(units);
};

const addUnit = async (req, res) => {
  const { bloodGroup, componentType, volumeMl, donorId } = req.body;
  const unitId = 'BLD-' + Date.now().toString().slice(-6);
  const expiryDate = calculateExpiryDate(componentType);

  const unit = await BloodInventory.create({
    unitId, bloodGroup, componentType, volumeMl: volumeMl || 450,
    status: 'quarantined', donorId, expiryDate
  });
  res.status(201).json(unit);
};
```

A7 Emergency Requisition Dispatch

```
const approveRequest = async (req, res) => {
  const request = await EmergencyRequest.findById(req.params.id);
  if (!request) return res.status(404).json({ message: 'Request not found' });

  const compatibleGroups = COMPATIBILITY_MAP[request.bloodGroup] || [request.bloodGroup];
  const matchingUnits = await BloodInventory.find({
    bloodGroup: { $in: compatibleGroups },
    componentType: request.componentType,
    status: 'available',
    expiryDate: { $gt: new Date() }
  }).sort({ expiryDate: 1 }).limit(request.unitsRequired);

  if (matchingUnits.length < request.unitsRequired) {
    return res.status(409).json({ message: 'Insufficient compatible stock available.' });
  }

  const allocatedIds = matchingUnits.map(u => u.unitId);
  await BloodInventory.updateMany(
    { _id: { $in: matchingUnits.map(u => u._id) } },
    { $set: { status: 'allocated' } }
  );

  request.status = 'approved';
  request.allocatedUnits = allocatedIds;
  await request.save();

  res.status(200).json({ message: 'Request approved and units allocated', request });
};
```

APPENDIX LABORATORY SCREENING

A8 Pathology Testing Routes

```
const express = require('express');
const router = express.Router();
const { protect, authorize } = require('../middleware/auth');

router.get('/pending', protect, authorize('admin'), async (req, res) => {
  const quarantinedUnits = await BloodInventory.find({ status: 'quarantined' })
    .sort({ collectedDate: 1 });
  res.status(200).json(quarantinedUnits);
});

router.post('/screen', protect, authorize('admin'), async (req, res) => {
  const { unitId, hiv, hbv, hcv, syphilis, malaria } = req.body;
  const unit = await BloodInventory.findOne({ unitId });
  if (!unit) return res.status(404).json({ message: 'Unit not found' });

  const reactive = hiv || hbv || hcv || syphilis || malaria;
  const outcome = reactive ? 'failed' : 'passed';

  await TestingLab.create({
    unitId: unit._id, hiv, hbv, hcv, syphilis, malaria,
    result: outcome, testedBy: req.user.name, testedAt: new Date()
  });

  unit.status = reactive ? 'discarded' : 'available';
  await unit.save();

  res.status(200).json({ status: unit.status, outcome });
});
```

A9 Component Stock Summary Aggregation

```
const getStockMatrix = async (req, res) => {
  const matrix = await BloodInventory.aggregate([
    { $match: { status: 'available', expiryDate: { $gt: new Date() } } },
    { $group: { _id: '$bloodGroup', totalUnits: { $sum: 1 } } }
  ]);
  res.status(200).json(matrix);
};
```

APPENDIX FRONTEND & VERCEL

A10 Client API Service (Axios Instance)

```
import axios from 'axios';

const API = axios.create({
  baseURL: '/api',
  headers: { 'Content-Type': 'application/json' }
});

API.interceptors.request.use((config) => {
  const token = localStorage.getItem('hemovault_token');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

export default API;
```

A11 Vercel Serverless Entry Point (api/index.js)

```
const { app } = require('../server/src/server');
const { connectDB } = require('../server/src/config/db');
const User = require('../server/src/models/User');
const { seedData } = require('../server/src/seeds/seed');

let isSeeded = false;

module.exports = async (req, res) => {
  try {
    await connectDB();
    if (!isSeeded) {
      const count = await User.countDocuments();
      if (count === 0) await seedData(false);
      isSeeded = true;
    }
  } catch (err) {
    console.error('Database connection error:', err.message);
  }
  return app(req, res);
};
```

SCREENSHOTS OF THE PROJECT

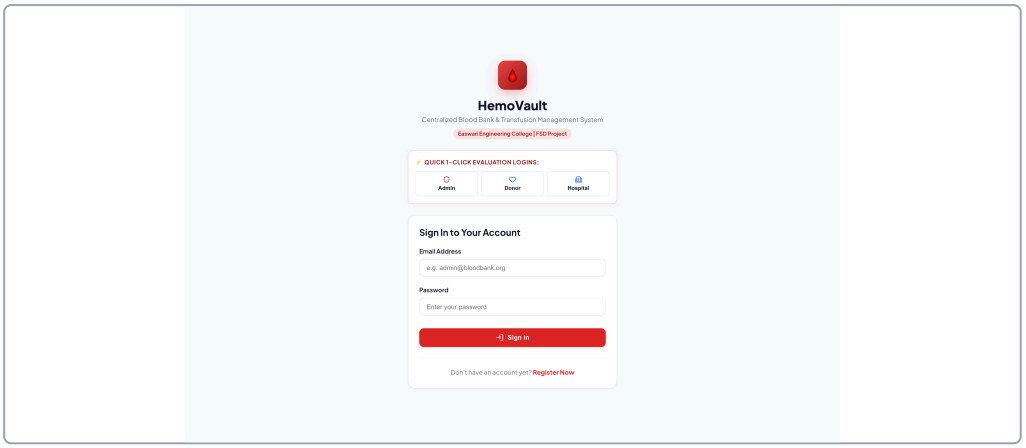


Figure S1 Sign-in page of the HemoVault Blood Bank Platform

The sign-in page collects user credentials and provides quick-access evaluation buttons for Administrator, Donor, and Hospital test accounts. Upon submission, the server returns a signed JWT token that activates the corresponding role portal.

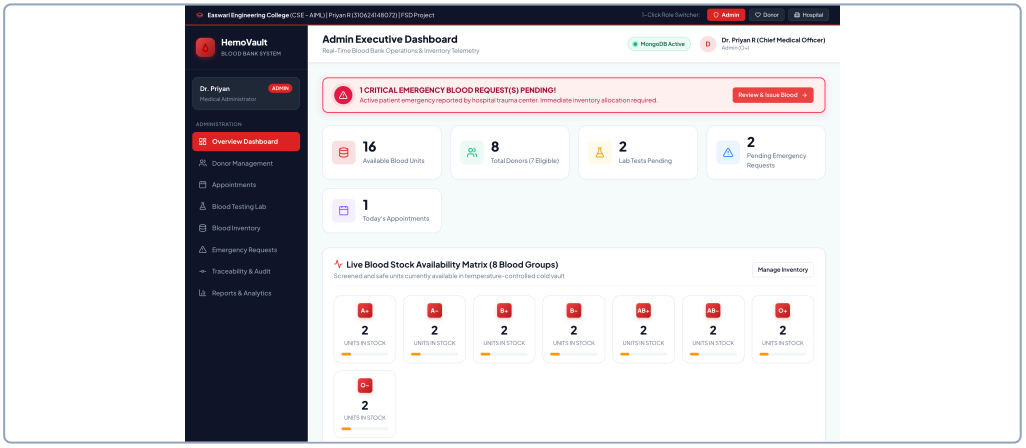


Figure S2 Admin Dashboard with statistics and 8-blood-group inventory matrix

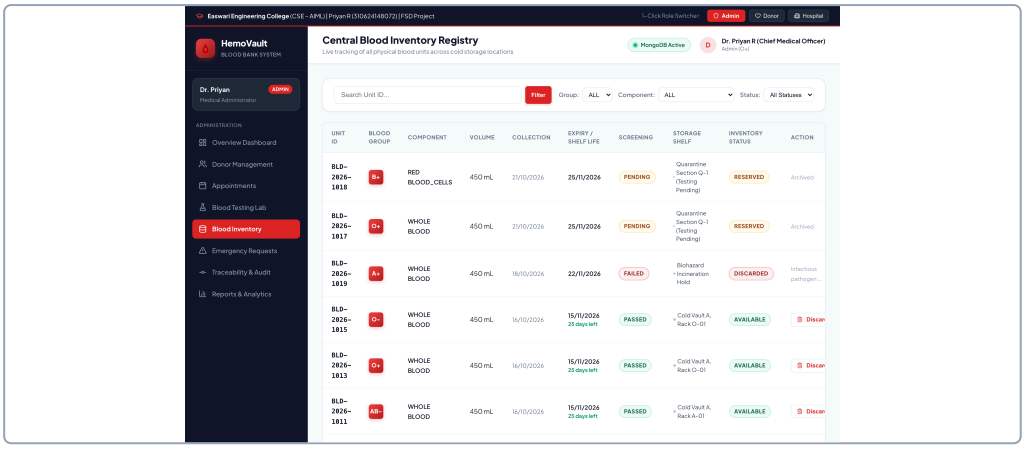


Figure S3 Blood inventory stock management & component distribution

The inventory hub displays all biological units in the system with filter controls for blood group, component type, and status. Operators can record new whole blood intakes and execute safe biohazard disposals.

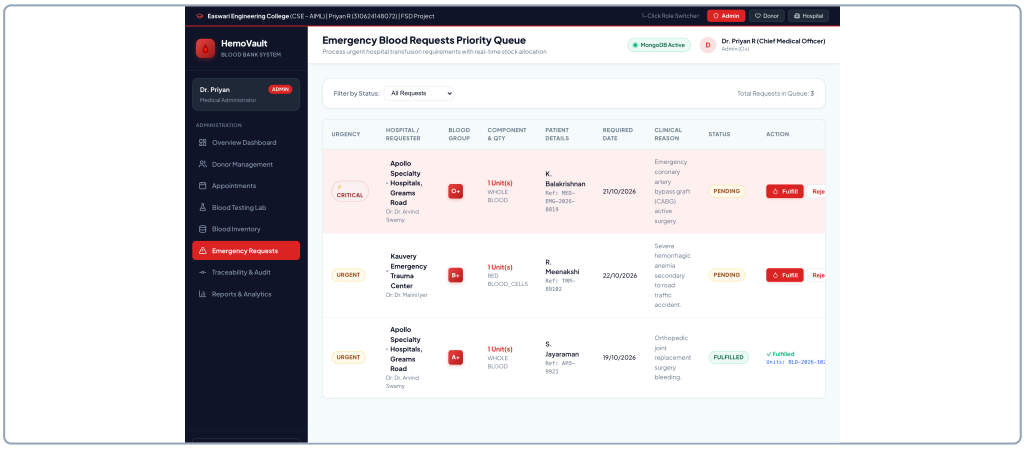


Figure S4 Hospital emergency requisition queue and compatibility matching

REFERENCES

[1] World Health Organization. *Requirements for the Collection, Processing and Quality Control of Blood, Blood Components and Plasma Derivatives*. WHO Technical Report Series, Geneva, 2021.

[2] National Blood Transfusion Council. *Standards for Blood Banks and Blood Transfusion Services*. Ministry of Health and Family Welfare, Government of India, New Delhi, 2022.

[3] Express.js Documentation. *Routing and Middleware Architecture*. https://expressjs.com/en/guide/routing.html, 2026.

[4] OWASP Foundation. *REST Security and Authentication Cheat Sheet Series*. https://cheatsheetseries.owasp.org/, 2025.

[5] MongoDB Documentation. *Schema Design, Document Indexing, and Transaction Management*. https://www.mongodb.com/docs/, 2026.

[6] Mongoose ODM. *Schema Validation, Middleware Hooks, and Population*. https://mongoosejs.com/docs/, 2026.

[7] React Documentation. *Components, Hooks, State Management, and Virtual DOM*. https://react.dev/, 2026.

[8] Vite Documentation. *Fast Frontend Tooling and Hot Module Replacement*. https://vite.dev/, 2026.

[9] Vercel Documentation. *Serverless Functions, Routing Rewrites, and Monorepo Deployments*. https://vercel.com/docs, 2026.

[10] JSON Web Token (JWT). *RFC 7519: JSON Web Token Architecture*. Internet Engineering Task Force (IETF), https://jwt.io/, 2024.

Online documentation and statutory guidelines were consulted in October 2026. Source code, schema models, and interface screenshots in this report were taken from the project repository and a live run of the HemoVault platform; the application can be reviewed at the GitHub link given in the appendix.